

Calibration Engine — Design Checkpoint

Yassine

August 10, 2026

Contents

1	Overview	3
2	Concepts Reference	3
2.1	Stochastic Simulation vs. Supervised Learning	3
2.2	Log Returns	3
2.3	Volatility	3
2.4	Annualization	3
2.5	Fit Metrics	3
3	Module: <code>utils.py</code>	4
3.1	Locked Decisions	4
4	Module: <code>base.py</code> — <code>CalibratorBase</code>	4
4.1	Locked Decisions	4
5	Model 1: GBM (Geometric Brownian Motion)	4
5.1	Model	4
5.2	Calibration	5
5.3	Locked Decisions	5
6	Model 2: Merton Jump-Diffusion	5
6.1	Model	5
6.2	Calibration Strategy	5
6.3	Mixture Likelihood	5
6.4	Locked Decisions	5
7	Model 3: Heston Stochastic Volatility	6
7.1	Model	6
7.2	Feller Condition	6
7.3	Current Calibration (Realized Vol)	6
7.4	Planned Upgrade: Implied Vol Surface Calibration (Post-M4)	6
7.5	Locked Decisions	7
8	MLflow Integration	7
8.1	Locked Decisions	7
9	M1 Extension: Options Chain Data	7
9.1	Data Model	7
9.2	Key Decisions	7
10	File Structure	7

11 Test Summary	8
12 Next: M3 — Simulation Engine	8

1 Overview

Milestone 2 (Calibration Engine) estimates model parameters from historical price data ingested in M1. Three stochastic models are supported: Geometric Brownian Motion (GBM), Merton Jump-Diffusion, and Heston Stochastic Volatility.

All design decisions below were finalized collaboratively and are ready for implementation handoff to Claude Code. M2 code has been implemented and reviewed — all 20 tests pass.

2 Concepts Reference

2.1 Stochastic Simulation vs. Supervised Learning

The models in this project (GBM, Heston, Jump-Diffusion) are **stochastic process models**, not supervised ML prediction models like XGBoost. They describe the statistical shape of how prices move (drift, volatility, jumps), and we generate thousands of possible futures by sampling from that shape. The distribution of outcomes is used to price options and compute risk metrics. This is fundamentally different from training a model to predict tomorrow's price.

2.2 Log Returns

Raw prices are not directly usable for modeling because a \$5 move means different things at different price levels. We convert prices to **log returns**:

$$r_t = \ln\left(\frac{S_t}{S_{t-1}}\right)$$

Log returns are additive over time (a weekly return is the sum of daily log returns) and symmetric (+10% and -10% cancel exactly). All three models are defined in terms of log returns.

2.3 Volatility

Volatility (σ) is the annualized standard deviation of log returns. It measures how much a stock's price swings. Low σ : steady, predictable. High σ : wild, unpredictable. It is the single most important parameter in options pricing.

2.4 Annualization

Daily statistics are scaled to annual terms using 252 trading days per year:

$$\mu_{\text{annual}} = \frac{\mu_{\text{daily}}}{dt}, \quad \sigma_{\text{annual}} = \frac{\sigma_{\text{daily}}}{\sqrt{dt}}, \quad dt = \frac{1}{252}$$

The square-root scaling for volatility comes from the assumption that daily returns are independent.

2.5 Fit Metrics

Log-likelihood

Measures how probable the observed data is under the fitted model. Higher is better. Only meaningful for *comparison* between models.

AIC Akaike Information Criterion: $2k - 2 \ln \hat{L}$. Penalizes model complexity (k = number of parameters). Lower is better. Prevents overfitting by taxing extra parameters.

BIC Bayesian Information Criterion: $k \ln n - 2 \ln \hat{L}$. Same idea as AIC but harsher penalty that scales with sample size n . Tends to favor simpler models.

Jarque–Bera

Tests whether residuals are normally distributed by checking skewness and kurtosis. Low p -value means residuals are *not* normal. GBM almost always fails this on real data, which motivates jump-diffusion and Heston.

AIC and BIC are used for **cross-model comparison**: calibrate all three models on the same stock, compare their AIC/BIC, and the lowest value indicates the best complexity-adjusted fit. This is why they are computed on the base class (consistent calculation) and logged to MLflow.

3 Module: `utils.py`**3.1 Locked Decisions**

- **Log returns over simple returns** — additive, symmetric, required by all three models.
- **NaN handling**: compute returns first, then drop NaN entries from the returns array. Do *not* forward-fill prices — this avoids manufacturing fake returns that the jump-diffusion calibrator could mistake for real jumps.
- `TRADING_DAYS_PER_YEAR = 252` as a module-level constant.
- `dt` as a function parameter (default `1/252`) for multi-frequency support (daily, weekly).
- Separate `annualize_mean()` and `annualize_volatility()` functions (not a combined tuple return). They operate on precomputed scalars, not raw arrays.
- `rolling_volatility()` for diagnostics and Heston calibration. Pure NumPy (uses `sliding_window_view`, no pandas dependency).

4 Module: `base.py` — `CalibratorBase`**4.1 Locked Decisions**

- **Stateful design**: `calibrate()` stores fitted parameters and residuals on `self`. `goodness_of_fit()` reads stored state instead of recomputing.
- `goodness_of_fit()` is a **concrete method** on the base class (not abstract). All fit metrics (log-likelihood, AIC, BIC, Jarque–Bera) are computed identically across models. The parameter count is derived from the Pydantic model via `len(type(self).params).model_fields)`.
- Guard: calling `goodness_of_fit()` before `calibrate()` raises `RuntimeError`.
- `_is_calibrated` flag tracks state.

5 Model 1: GBM (Geometric Brownian Motion)**5.1 Model**

$$dS = \mu S dt + \sigma S dW$$

Two parameters: drift μ and volatility σ , both constant. Assumes smooth price movements and constant volatility. Good baseline, but underprices options because it underestimates extreme moves.

5.2 Calibration

$$\sigma = \text{annualize_volatility}(\text{std}(r_t, \text{ddof} = 1), dt) \quad (1)$$

$$\mu = \text{annualize_mean}(\text{mean}(r_t), dt) + \frac{1}{2}\sigma^2 \quad (2)$$

5.3 Locked Decisions

- `GBMParams(mu: float, sigma: float)` — plain Pydantic model.
- `ddof=1` for sample standard deviation.
- Itô correction ($+\frac{1}{2}\sigma^2$) applied to drift.
- Rolling window slicing is the **caller's responsibility**, not the calibrator's. The calibrator fits whatever prices it receives.

6 Model 2: Merton Jump-Diffusion

6.1 Model

$$\frac{dS}{S} = (\mu - \lambda k) dt + \sigma dW + J dN$$

Extends GBM with Poisson-distributed jumps. Five parameters: μ, σ (diffusion), λ (jump intensity), μ_j, σ_j (jump size distribution). Better at capturing crash risk and earnings surprises than GBM.

6.2 Calibration Strategy

Hybrid approach — uses GBM as a baseline (a design decision originating from the insight that GBM models the “normal” regime, making it a cleaner threshold baseline than raw statistics contaminated by jumps):

1. Receive pre-calibrated GBM params (decoupled — calibrators don't depend on each other internally). Falls back to internal GBM calibration if not provided.
2. Use GBM estimates as threshold to flag jumps: $|r_t - \mu dt| > 3\sigma\sqrt{dt}$.
3. Compute initial λ, μ_j, σ_j from flagged jumps.
4. Refine all 5 parameters via MLE on the mixture distribution using `scipy.optimize.minimize` (L-BFGS-B).

6.3 Mixture Likelihood

On any day, the return pdf is:

$$(1 - \lambda dt) \cdot \mathcal{N}(\mu dt, \sigma^2 dt) + \lambda dt \cdot \mathcal{N}(\mu dt + \mu_j, \sigma^2 dt + \sigma_j^2)$$

6.4 Locked Decisions

- `JumpDiffusionParams(mu, sigma, lambda_j, mu_j, sigma_j)`.
- `calibrate()` receives `gbm_params: GBMParams` as input (decoupled).
- MLE is primary; threshold method generates initial guesses only.
- If MLE converges to $\lambda \approx 0$ (no jumps), return as-is. AIC/BIC comparison will correctly favor plain GBM in this case.

7 Model 3: Heston Stochastic Volatility

7.1 Model

$$dS = \mu S dt + \sqrt{v} S dW_S \quad (3)$$

$$dv = \kappa(\theta - v) dt + \xi \sqrt{v} dW_v \quad (4)$$

$$\text{corr}(dW_S, dW_v) = \rho \quad (5)$$

Volatility itself follows a mean-reverting random process. Five parameters:

θ Long-run variance level (mean-reversion target).

κ Mean-reversion speed (“spring stiffness” — high κ snaps volatility back quickly after a spike).

ξ Vol-of-vol (how wildly volatility itself fluctuates).

ρ Price-vol correlation (usually negative: the “fear effect” where drops cause volatility spikes).

v_0 Current variance level.

7.2 Feller Condition

$$2\kappa\theta > \xi^2$$

Ensures variance never hits zero. Strategy: constrain the optimizer (SLSQP) as primary. If constrained fit is significantly worse ($> 2\times$ loss), fall back to unconstrained (L-BFGS-B) with a warning flag.

7.3 Current Calibration (Realized Vol)

Fits against the **realized volatility term structure**: historical volatility computed at horizons [5, 21, 63, 126, 252] days. Uses the Heston expected variance formula:

$$\mathbb{E}[v_t] = \theta + (v_0 - \theta) e^{-\kappa t}$$

Minimize sum of squared errors between model-implied and realized vols.

Known limitation: this formula only involves κ, θ, v_0 . The parameters ξ (vol-of-vol) and ρ (price-vol correlation) are underdetermined because they do not appear in the expected variance equation. They remain near initial guesses.

7.4 Planned Upgrade: Implied Vol Surface Calibration (Post-M4)

The proper fix is to calibrate against the **implied volatility surface** from options market data (which M1 now ingests). The skew shape across strikes at a given expiry is driven by ξ and ρ , giving the optimizer signal for all 5 parameters. This requires:

1. Heston characteristic function pricer (numerical Fourier inversion) — new code, ~100 lines.
2. Black-Scholes implied vol inversion (root-finder) — needed by M4 anyway.
3. Second calibration path in **HestonCalibrator**: if options surface data is provided, fit against it; otherwise fall back to realized vol.

Scheduled for after M4 when the pricing infrastructure is in place.

7.5 Locked Decisions

- `HestonParams(kappa, theta, xi, rho, v0, feller_satisfied: bool)`.
- Rolling volatility from `utils.py` provides the fitting targets (current approach).
- Constrained optimization primary, unconstrained fallback with warning flag.
- Standalone `feller_condition_satisfied()` helper exposed.

8 MLflow Integration

8.1 Locked Decisions

- **External wrapper pattern:** calibrators know nothing about MLflow. All logging happens at the call site. Follows the project’s core principle of decoupling responsibilities for testability and debugging.
- Logged per run: ticker, date range, model type, fitted parameters, fit metrics (log-likelihood, AIC, BIC, Jarque–Bera), residual distribution plot.
- Experiment naming: `calibration/{ticker}`.
- Same pattern carries forward to M3 (simulation) and M4 (pricing).

9 M1 Extension: Options Chain Data

Added to support future Heston surface calibration. Implemented and reviewed.

9.1 Data Model

`OptionData`: ticker, expiration_date, strike, option_type (call/put), bid, ask, last_price, implied_volatility, volume (nullable), open_interest (nullable), fetched_at.

9.2 Key Decisions

- **Snapshot model, not append-only:** options prices change daily. Each fetch pulls the full current chain stamped with `fetched_at = date.today()`.
- Composite PK: (ticker, expiration_date, strike, option_type, fetched_at) — keeps daily snapshots.
- NaN implied vol → skip row (unusable for calibration). Zero bid/ask → keep (legitimate for deep OTM contracts). NaN bid/ask → normalize to 0.0.
- `load_options_chain()` defaults to most recent snapshot via a scoped scalar subquery on `max(fetched_at)`.

10 File Structure

```
src/calibration/
__init__.py
base.py           # CalibratorBase (stateful, concrete goodness_of_fit)
gbm.py           # GBMCalibrator, GBMParams
jump_diffusion.py # JumpDiffusionCalibrator, JumpDiffusionParams
```

```

heston.py          # HestonCalibrator, HestonParams
utils.py          # log_returns, annualize_*, rolling_volatility

src/data/         # M1 (extended)
models.py         # StockData, OptionData, TickerMetadata
storage.py        # ohlcv + options_chain tables, CRUD
fetcher.py        # OHLCV append-only + options snapshot fetch

```

11 Test Summary

File	Tests	Status
test_calibration.py	20	All pass
test_options_data.py	20	18 pass, 2 require Docker/network
test_storage.py	10	Require Docker
test_fetcher.py	2	Require network

Total: 52 tests collected, all non-infra tests pass.

12 Next: M3 — Simulation Engine

Generate Monte Carlo price paths for all 3 models. Key topics to design: `SimulatorBase` interface, vectorized GBM/Heston/Jump-Diffusion path generation, variance reduction techniques (antithetic variates, stratified sampling), seed management, and performance targets.